大模型推理调度系统技术报告

基于 SGLang 的低延迟、SLA 感知在线推理服务

Team Alpha

2026 年 4 月

目录

1 系统概述 **2**
1.1 总体架构 2
1.2 任务类型与 SLA 等级 3

2 环境搭建与服务启动 **3**
2.1 依赖安装（`setup.sh`） 3
2.2 服务启动（`run.sh`） 4
2.2.1 GPU 架构自动检测 4
2.2.2 SGLang 后端启动 5

3 调度策略：准入控制 **5**
3.1 整体流程 5
3.2 EWMA 延迟追踪 5
3.3 队列深度感知的准入公式 6
3.4 探针机制（防死锁） 6

4 任务调度执行 **7**
4.1 优先级队列 7
4.2 Dispatcher 协程 7
4.3 任务生命周期（`handle_task`） 7
4.4 SGLang 内部优先级透传 8

5 推理加速技术 **8**
5.1 张量并行（Tensor Parallelism） 8
5.2 连续批处理（Continuous Batching） 8
5.3 Chunked Prefill（分块预填充） 9
5.4 RadixAttention（前缀缓存复用） 9
5.5 GPU 架构兼容（Blackwell 适配） 9

6 实时监控（Dashboard） **9**

7 关键设计决策总结 **10**

1 系统概述

本系统围绕 **在线推理服务的 SLA（服务级别协议）达成率最大化**这一核心目标设计，面向多任务类型、多 SLA 等级的竞赛评测场景。系统以 SGLang 作为底层推理引擎，在其之上实现了一套自适应调度层，负责任务的准入控制、优先级排队与并发推理编排。

1.1 总体架构

```
┌──────────────────────────────────────────────────────┐
│                   评测平台 (Platform)                 │
│         /query   /ask   /submit   /register          │
└──────────────────────┬───────────────────────────────┘
                       │ HTTP
┌──────────────────────▼───────────────────────────────┐
│              调度客户端 (contestant.main)              │
│                                                      │
│  ┌──────────────┐   ┌──────────────────────────┐     │
│  │  Poller Loop │──►│  Scheduler.should_accept  │     │
│  │  (query→ask) │   │  EWMA × (1 + W/R)         │     │
│  └──────────────┘   └──────────────┬───────────┘     │
│                                     │ accept          │
│                     ┌───────────────▼──────────┐      │
│                     │  asyncio.PriorityQueue   │      │
│                     │  (高价值任务优先出队)      │      │
│                     └──────────────┬───────────┘      │
│                                    │                  │
│                     ┌──────────────▼──────────┐       │
│                     │  Dispatcher Coroutine   │       │
│                     │  (并发 handle_task)      │       │
│                     └──────────────┬──────────┘       │
└────────────────────────────────────┼─────────────────┘
                                     │ HTTP
┌────────────────────────────────────▼─────────────────┐
│              SGLang 推理后端 (localhost:30000)         │
│                                                      │
```

```
│  Tensor Parallel (tp=4)   ·  Continuous Batching      │
│  Chunked Prefill (4096)   ·  RadixAttention          │
│  Priority Scheduling      ·  Triton/PyTorch Backend  │
└──────────────────────────────────────────────────────┘
```

1.2 任务类型与 SLA 等级

系统支持三种评测任务类型：

任务类型	说明	计分权重
`generate_until`	自回归文本生成，遇停止词截断	2.0×
`loglikelihood`	计算条件对数概率 P(续写 | 提示)	1.0×
`loglikelihood_rolling`	计算整段文本的总对数似然	1.0×

SLA 等级按 TTFT（首 Token 时延）划分，共 8 级：

SLA 等级	TTFT 上限	得分权重
Bronze	10.0 s	1.0×
Silver	8.0 s	1.2×
Gold	6.0 s	1.5×
Platinum	4.0 s	1.7×
Diamond	2.0 s	2.0×
Stellar	1.5 s	2.2×
Glorious	0.8 s	2.4×
Supreme	0.5 s	2.5×

---

2 环境搭建与服务启动

2.1 依赖安装（`setup.sh`）

平台在首次部署时执行 `contestant/setup.sh`，结果被缓存，后续重启不再重复执行：

```
# 1. 创建隔离虚拟环境
python3 -m venv .venv && source .venv/bin/activate
```

```
# 2. 安装 Python 依赖
pip install -r contestant/requirements.txt

# 3. 从本地 clone 安装 SGLang（放宽 cuda-python 版本约束以兼容 CUDA 12.8）
sed -i 's/cuda-python==12\.9/cuda-python>=12.8/' sglang/python/pyproject.toml
pip install -e "sglang/python/"

# 4. 可选：安装 flash-attn 提升吞吐（失败不影响运行）
pip install flash-attn --no-build-isolation
```

> **注意**：SGLang 上游硬依赖 `cuda-python==12.9`，而竞赛服务器为 CUDA 12.8。通过放宽为 `>=12.8` 的下界约束，避免了安装失败，两者 API 完全兼容。

2.2 服务启动（run.sh）

`contestant/run.sh` 负责每次启动时的完整流程：

2.2.1 GPU 架构自动检测

```
# 检测注意力后端
ATTN_BACKEND=$(python3 - <<'EOF'
import torch
try:
    major, _ = torch.cuda.get_device_capability(0)
    print("triton" if major >= 12 else "flashinfer")
except Exception:
    print("triton")   # 查询失败 → Blackwell (SM 12.x) on CUDA < 12.9
EOF
)

# 采样后端与注意力后端保持一致
SMPL_BACKEND=$( [ "${ATTN_BACKEND}" = "triton" ] && echo "pytorch" || echo "flashinfer" )
```

RTX 5090（Blackwell，SM 12.0）在 CUDA 12.8 下 `torch.cuda.get_device_capability()` 返回异常，系统捕获该异常并自动切换到 Triton 注意力后端与 PyTorch 采样后端，绕过 FlashInfer JIT 对 SM 12.x 的不兼容问题。

2.2.2 SGLang 后端启动

```
python -m sglang.launch_server \
    --model-path "${MODEL_PATH}" \
    --host 0.0.0.0 --port 30000 \
    --tp-size "${TP_SIZE}" \
    --schedule-policy fcfs \
    --enable-priority-scheduling \
    --chunked-prefill-size 4096 \
    --attention-backend "${ATTN_BACKEND}" \
    --sampling-backend "${SMPL_BACKEND}" &
```

SGLang 就绪检测（最多等待 55 秒）后，启动调度客户端：

```
SGLANG_URL=http://localhost:30000 python -m contestant.main
```

---

3 调度策略：准入控制

调度层的核心问题是：**对于每一个 `query` 返回的任务概览，是否调用 `ask` 接受该任务？**

3.1 整体流程

```
query() → overview
    |
    ▼
should_accept(overview)?
    ├── NO  → 丢弃，立刻 re-query（循环）
    └── YES → ask(task_id, sla) → 获取完整任务数据
                  |
                  ▼
            放入 PriorityQueue
```

3.2 EWMA 延迟追踪

系统对每个 `(task_type, sla)` 组合独立维护一个指数加权移动平均（EWMA）延迟估计，衰减系数 $= 0.3$：

$$\mathrm{EWMA}_t = 0.3 \times \mathrm{elapsed}_t + 0.7 \times \mathrm{EWMA}_{t-1}$$

同时维护最近 50 个样本的滑动窗口，用于计算 P95 延迟、平均延迟与 SLA 命中率，供 Dashboard 实时展示。

冷启动处理：EWMA 为 None 时（无历史数据），跳过延迟检查，仅受并发上限约束，以”探针”方式快速建立初始估计。

3.3 队列深度感知的准入公式

$$\hat{T} = \mathrm{EWMA}_{(\mathrm{type,sla})} \times \left(1 + \frac{W}{R}\right)$$

- $W =$ `num_waiting_reqs`：SGLang 内部排队等待 GPU 的任务数（实时，每 0.5 s 从 `/server_info` 刷新）
- $R =$ `num_running_reqs`：当前正在 GPU 上运行的任务数
- W/R（queue_factor）为排队理论中的等待倍数：若 $W = 4$、$R = 2$，新任务预计等待约 2 个推理周期

准入判断：

$$\mathrm{accept} \iff \hat{T} < \mathrm{SLA_TTFT[sla]}$$

对比旧方案（load_ratio）：

方案	公式	问题
旧：load_ratio	`ewma × (1 + active_count/24)`	固定阈值，对任务长度盲目
新：queue_factor	`ewma × (1 + W/R)`	实时感知，长任务自动收紧

新方案的核心优势在于**任务长度感知**：`generate_until` 等耗时任务会占据更多运行时间，使 R 饱和、W 积压，令 W/R 自然升高，无需人工调参即可自动收紧准入。

3.4 探针机制（防死锁）

若某个 `(task_type, sla)` 组合被连续拒绝 8 次（`PROBE_THRESHOLD`），强制接受一个任务以刷新 EWMA，防止因 EWMA 永久过时导致该类任务被永久屏蔽。

4 任务调度执行

4.1 优先级队列

接受的任务进入 `asyncio.PriorityQueue`，优先级分数为：

$$\text{priority} = -(\text{sla_weight} \times \text{task_weight})$$

取负数使 Python 小根堆等效为大根堆，高价值任务（Supreme + generate_until）优先出队。

任务类型	SLA 等级	综合优先级得分
generate_until	Supreme	−5.0（最高）
generate_until	Bronze	−2.0
loglikelihood	Supreme	−2.5
loglikelihood	Bronze	−1.0（最低）

4.2 Dispatcher 协程

专用 `dispatcher` 协程持续从优先级队列出队，为每个任务创建独立的 `asyncio.Task`：

```python
async def dispatcher(task_queue, ...):
    while not stop_event.is_set() or not task_queue.empty():
        item = await asyncio.wait_for(task_queue.get(), timeout=0.1)
        asyncio.create_task(handle_task(...))
```

这使任务的排队（优先级决策）与执行（推理）完全解耦，dispatcher 不阻塞任何推理操作。

4.3 任务生命周期（`handle_task`）

```
ask() 返回完整任务
    |
    ▼
asyncio.gather(*[process_one(msg) for msg in messages])
    |   ← 同一任务的所有消息并发发往 SGLang
    ▼
submit(result_messages)
    |
    ▼
```

```
scheduler.latency.record(task_type, sla, elapsed)
```

超期仍提交：即便超过 SLA 时限，系统依然提交结果（得 0 分），避免触发 600 s 硬超时的 $-2\times$ 惩罚。这是一个关键的风险对冲设计：

$$\text{SLA miss} \Rightarrow 0\ \text{分} \quad \gg \quad \text{Hard timeout} \Rightarrow -2 \times \text{reward 扣罚}$$

4.4 SGLang 内部优先级透传

每个推理请求携带 `priority` 字段（0–7），与 SLA 等级一一对应：

SLA	SGLang priority
Bronze	0
Supreme	7

配合 `--enable-priority-scheduling`，SGLang 内部调度器优先处理高 priority 请求，实现端到端的优先级保障。

---

5 推理加速技术

5.1 张量并行（Tensor Parallelism）

针对 Qwen3-32B 模型（~64 GB 参数量），采用 4 卡张量并行（`--tp-size 4`）：

- 每张 GPU 存储约 1/4 的模型权重（~16 GB）
- 每次 forward pass 通过 NCCL All-Reduce 同步各卡的中间激活
- 单卡显存占用从 64 GB 降至 ~16 GB，4 张 RTX 5090 完整容纳模型

5.2 连续批处理（Continuous Batching）

SGLang 实现了 Iteration-level 连续批处理（Orca 论文方案）：

- 每个 decode 步骤，所有正在生成的序列被合并为一个 batch 同时计算
- 新完成的序列立即释放 KV Cache 槽位，新请求无需等待整个 batch 完成
- 相比静态批处理，显著提高 GPU 利用率和系统吞吐量

5.3 Chunked Prefill（分块预填充）

长 prompt 的 prefill 阶段拆分为固定大小的 chunk（4096 tokens/chunk）：

- 每个 chunk 间隙，decode 请求可插入执行，避免长 prefill 独占 GPU
- 与 `--enable-priority-scheduling` 协同：高优先级任务可在 chunk 间隙插队
- 对 Supreme（0.5 s SLA）等紧急任务，显著降低因其他任务长 prefill 导致的队列阻塞时延

5.4 RadixAttention（前缀缓存复用）

SGLang 的 RadixAttention 将 KV Cache 组织为前缀树结构：

- 相同前缀的请求（如同一 benchmark 的不同选项）共享 KV Cache
- `loglikelihood` 任务通常有相同 prompt + 不同 continuation，同一任务的多条消息通过 `asyncio.gather` 并发发送，RadixAttention 自动复用 prompt 部分的 KV Cache，4 个选项仅需计算一次 prompt prefill

5.5 GPU 架构兼容（Blackwell 适配）

RTX 5090（SM 12.0 / Blackwell）在 CUDA 12.8 下存在两处 FlashInfer JIT 不兼容：

问题	触发位置	解决方案
注意力 JIT 编译失败	`flashinfer/jit/core.py`	`--attention-backend triton`
采样 JIT 编译失败	`flashinfer/sampling.py`	`--sampling-backend pytorch`

两处独立 fallback 在 `run.sh` 中自动检测并启用，无需人工干预，保证 Blackwell 架构下的完整功能。

---

6 实时监控（Dashboard）

系统内置实时 Dashboard，基于 `rich` 库渲染，每 0.5 s 刷新：

- **任务统计**：已接受 / 已拒绝 / 已完成 / SLA miss 计数
- **延迟分布**：按 `(task_type, sla)` 分组展示 EWMA、P95 延迟、SLA 命中率
- **SGLang 队列**：`num_waiting_reqs` / `num_running_reqs` 实时值（0.5 s 轮询）
- **吞吐曲线**：近期任务完成率时序图
- **得分追踪**：从平台 `/scores` 端点实时拉取当前累计得分

SGLang 队列深度从每 2 s 轮询改为每 0.5 s 轮询，确保调度决策使用的 W/R 信号足够新鲜。

---

7 关键设计决策总结

决策	选择	原因
准入控制信号	SGLang `W/R` queue_factor	实时、任务长度感知，优于逻辑并发比
EWMA 分桶粒度	`(task_type, sla)`	不同类型/SLA 的延迟分布差异大，共用一个 EWMA 会产生偏差
排队策略	asyncio.PriorityQueue	确保高价值任务优先进入 SGLang，而非 FCFS
超期任务处理	依然提交	避免 $-2\times$ 硬超时惩罚，净收益为正
冷启动处理	探针：EWMA=None 时全接受	快速建立历史数据，避免初期过度保守
GPU 后端选择	自动检测，Blackwell 回退 Triton	保证跨架构兼容，无需手动配置

报告截止于 2026 年 4 月，对应代码版本 `29792b9`。